functable

Sign, variation and value tables for French mathematics

Version 0.1.0 — Nathan Scheinmann — 2026-07-14

Contents

1. Overview ... 2
2. `sign-table` ... 2
   2.1. Basic usage ... 2
   2.2. Parameters ... 2
      2.2.1. Factor dictionary keys ... 4
      2.2.2. Zero dictionary keys ... 4
   2.3. Direct signs — tables without factors ... 4
   2.4. Convexity display modes ... 5
   2.5. Variation values ... 6
   2.6. HD (hors-domaine) intervals ... 6
   2.7. Poles (valeurs interdites) ... 7
   2.8. Auto-sign inference with `fn` ... 7
   2.9. Simple table without factor strip ... 8
   2.10. Coloured background ... 9
   2.11. Domain boundary (range not in domain) ... 9
3. `fun-table` ... 11
   3.1. Basic usage ... 11
   3.2. Parameters ... 11
      3.2.1. Row dictionary keys ... 11
   3.3. Auto-computed values ... 11
   3.4. Custom display with dictionary x-values ... 12

1. Overview

`functable` provides two functions for drawing the standard French high-school mathematics tables:

- **`sign-table`** — tableau de signes et de variations (sign/variation/convexity tables in the *tkz-tab* style)
- **`fun-table`** — tableau de valeurs (function value table)

Both functions support auto-computation: provide a Typst closure and the values or signs are computed automatically.

2. `sign-table`

2.1. Basic usage

A sign table contains a header row (x), one or more **factor rows** whose signs multiply to give the **summary row**, an optional **variation row** with diagonal arrows, and optionally a second block for f’‘ with a **convexity row**.

```
#sign-table(
  factors: (
    (expr: $x - 1$, zeros: ((value: $1$, approx: 1),), signs: ("-", "+")),
    (expr: $x + 2$, zeros: ((value: $-2$, approx: -2),), signs: ("-", "+")),
  ),
  summary-label: $f'(x)$,
  variation: true,
  variation-label: $f(x)$,
)
```

	x	$-\infty$		-2		1		$+\infty$
facteurs	$x-1$		$-$		$-$	0	$+$	
	$x+2$		$-$	0	$+$		$+$	
	$f'(x)$		$+$	0	$-$	0	$+$	
	$f(x)$		↗		↘		↗	

2.2. Parameters

Parameter	Type	Default	Description
`factors`	`array`	`()`	Factor rows (see factor keys below).
`signs`	`array`	`()`	Direct signs for the summary row, without factor rows. Mutually exclusive with `factors`. Same values as a factor’s `signs`.
`zeros`	`array`	`()`	Zeros attached to top-level `signs` / `second-signs`. Same

			format as a factor’s `zeros`, including the number shorthand.
`summary-label`	`content`, `none`	`none`	Label for the f’ summary row.
`variation`	`bool`	`false`	Show variation row with diagonal arrows.
`variation-label`	`content`, `none`	`none`	Label for the variation row.
`variation-values`	`array`	`()`	Values on the variation row: `(at, label, pos)`.
`bounds`	`auto`, `none`, `dict`	`auto`	Domain bounds in x header. `auto` ⟶ $-\infty/+\infty$. `none` hides both. Dict: `(left: ..., right: ...)`.
`start-value`	`content`, `none`	`none`	Function value at the left edge of the variation row.
`start-pos`	`string`	`"auto"`	`"top"`, `"bottom"`, or `"auto"`.
`end-value`	`content`, `none`	`none`	Function value at the right edge of the variation row.
`end-pos`	`string`	`"auto"`	`"top"`, `"bottom"`, or `"auto"`.
`col-width`	`length`	`1.5cm`	Width of each interval column.
`row-height`	`length`	`0.9cm`	Height of sign/factor rows.
`var-row-height`	`auto`, `length`	`auto`	Height of variation and convexity rows. `auto` grows to fit the tallest label.
`second-factors`	`array`	`()`	Factor rows for f’‘. Same structure as `factors`.
`second-signs`	`array`	`()`	Direct signs for the f’‘ block, without factor rows. Mutually exclusive with `second-factors`. Zeros come from top-level `zeros`.
`second-summary-label`	`content`, `none`	`none`	Label for the f’‘ summary row.
`convexity`	`bool`, `string`	`false`	Convexity row. `true`/`"symbol"` ⟶ ∪/∩; `"text"` ⟶ “convexe”/“concave”; `"both"` ⟶ symbol above the word.
`convexity-label`	`content`, `none`	`none`	Label for the convexity row.
`hd-fill`	`color`	`rgb("#cfe2f3")`	Fill color when `hd-style: "fill"`.
`hd-style`	`string`	`"hatch"`	HD band style: `"hatch"`, `"fill"`, or `"blank"`.
`show-facteurs`	`bool`	`true`	Show rotated “facteur(s)” strip at the left spanning the factor rows.
`background`	`color`	`white`	Background colour for label knockout rectangles. Set to match your page or container fill.

2.2.1. Factor dictionary keys

Each entry in `factors` or `second-factors` is a dictionary. **Exactly one of `signs` or `fn` is required**; every other key is optional:

Parameter	Type	Default	Description
`expr`	`content`	—	*Optional.* Math expression for the row label (e.g. `$x - 1$`).
`zeros`	`array`	`()`	*Optional.* Zero declarations (see zero keys below).
`signs`	`array`	—	**Required unless `fn` is given.** Sign per interval: `"+"`, `"-"`, `""` (blank), `"HD"`. Takes precedence over `fn`.
`fn`	`function`	—	**Required unless `signs` is given.** `x => number` for auto-sign inference. Return `none` to mark HD.
`interdit`	`bool`	`false`	*Optional.* All zeros of this factor are forbidden values of f (shorthand for `pole: true` on each zero).

2.2.2. Zero dictionary keys

Each entry in a `zeros` array is either a **plain number** — shorthand for `(value: $n$, approx: n)`, convenient when the zero needs no special display — or a dictionary with the following keys:

Parameter	Type	Default	Description
`value`	`content`	—	**Required.** Displayed label (e.g. `$1$`, `$sqrt(2)$`).
`approx`	`float`	—	**Required.** Numeric approximation, used for sorting, merging shared zeros, and test-point computation.
`pole`	`bool`	`false`	*Optional.* Valeur interdite: double bar ‖ in summary/variation/convexity rows; breaks arrows.
`mark`	`content`, `"bar"`, `"hd"`	—	*Optional.* Custom marker in factor and summary rows. `"bar"` draws a double bar; `"hd"` continues the HD hatch through the point.
`summary-mark`	`content`, `"bar"`, `"hd"`	—	*Optional.* Custom marker in the summary row only.

The number shorthand and the dictionary form can be mixed freely:

```
zeros: (-2, (value: $sqrt(2)$, approx: calc.sqrt(2)), 3)
```

2.3. Direct signs — tables without factors

When the factorisation is not the point of the exercise, give the signs of the function directly with the top-level `signs` and `zeros` parameters instead of `factors` (the two are mutually exclusive):

```
#sign-table(
  signs: ("+", "-", "+"),
  zeros: (-2, 1),
  summary-label: $f(x)$,
)
```

x	$-\infty$		-2		1		$+\infty$
$f(x)$		$+$	0	$-$	0	$+$	

The same mechanism gives a **variation table alone**, with no sign row at all — leave `summary-label` at `none` and the signs only steer the arrows:

```
#sign-table(
  signs: ("-", "+"),
  zeros: (2,),
  variation: true,
  variation-label: $f(x)$,
  start-value: $+oo$,
  end-value: $+oo$,
  variation-values: ((at: 2, label: $-3$),),
)
```

x	$-\infty$		2		$+\infty$
$f(x)$	$+\infty$	$\searrow$	-3	$\nearrow$	$+\infty$

Per-zero options work as usual — mix the number shorthand with dictionaries to mark a pole:

```
signs: ("+", "-", "+"),
zeros: (-1, (value: $2$, approx: 2, pole: true)),
```

Symmetrically, `second-signs` drives the f’‘ block (and hence the convexity row) without `second-factors`; its zeros also come from the top-level `zeros` parameter. Note that a sign table **without** a variation table has always been the default — just leave `variation: false`.

2.4. Convexity display modes

The `convexity` parameter accepts, besides `false`, three display modes: `"symbol"` (equivalent to `true`) for the classic ∪/∩, `"text"` for the words “convexe”/“concave”, and `"both"` for the symbol stacked above the word:

```
#sign-table(
  second-signs: ("-", "+"),
  zeros: (0,),
  second-summary-label: $f''(x)$,
  convexity: "both",
  convexity-label: $f(x)$,
)
```

x	$-\infty$		0		$+\infty$
$f''(x)$		$-$	0	$+$	
$f(x)$		$\cap$ concave		$\cup$ convexe	

2.5. Variation values

Use `variation-values` to pin function values at interior zeros on the variation row:

```
variation-values: (
  (at: 2, label: $-3$),          // auto-positioned
  (at: 5, label: $0$, pos: "top"),    // forced top
)
```

`pos` accepts `"top"`, `"bottom"`, `"arrow"` (centred on the arrow line), or `"auto"`.

2.6. HD (hors-domaine) intervals

Mark an interval as `"HD"` in `signs` (or return `none` from `fn`) for intervals where the expression does not exist. The interval renders as a hatched band:

```
#sign-table(
  factors: (
    (
      expr: $1 / (2 sqrt(x))$,
      zeros: ((value: $0$, approx: 0, mark: "hd"),),
      fn: x => if x <= 0 { none } else { 1 / (2 * calc.sqrt(x)) },
    ),
  ),
  summary-label: $f'(x)$,
  variation: true,
  variation-label: $f(x)$,
  bounds: (left: none, right: $+oo$),
  start-value: $0$, start-pos: "bottom",
)
```

	x		0		$+\infty$
fact.	$\frac{1}{2\sqrt{x}}$			$+$	
	$f'(x)$			$+$	
	$f(x)$		0	$\nearrow$	

`mark: "hd"` continues the hatch **through** a boundary point — use this for a domain edge where f' is undefined but f continues (e.g. a vertical tangent), as opposed to `mark: "bar"` which draws an asymptote double bar.

2.7. Poles (valeurs interdites)

Set `pole: true` on a zero to draw a double bar ‖ in the summary, variation and convexity rows, and to break the variation arrows at that point. Use the `interdit: true` shorthand on a factor when all its zeros are forbidden values of f:

```
#sign-table(
  factors: (
    (expr: $x + 1$, zeros: ((value: $-1$, approx: -1),), signs: ("-", "+")),
    (
      expr: $x - 2$,
      zeros: ((value: $2$, approx: 2, pole: true),),
      signs: ("-", "+"),
    ),
  ),
  summary-label: $f(x)$,
  variation: true,
  variation-label: $f(x)$,
)
```

	x	$-\infty$		-1		2		$+\infty$
facteurs	$x+1$		$-$	0	$+$		$+$	
	$x-2$		$-$		$-$	0	$+$	
	$f(x)$		$+$	0	$-$	‖	$+$	
	$f(x)$		↗		↘	‖	↗	

2.8. Auto-sign inference with `fn`

When `signs` is omitted and `fn` is provided, the sign in each interval is inferred by evaluating the function at a test point (midpoint of adjacent zeros for interior intervals, offset from the boundary zero for outer intervals):

```
// f(x) = x³ − 3x:  f'(x) = 3(x+1)(x−1),  f''(x) = 6x
#sign-table(
  factors: (
    (expr: $x + 1$, zeros: ((value: $-1$, approx: -1),), fn: x => x + 1),
    (expr: $x - 1$, zeros: ((value: $1$,  approx:  1),), fn: x => x - 1),
  ),
  summary-label: $f'(x)$,
  variation: true,
  variation-label: $f(x)$,
  start-value: $+oo$,
  end-value: $+oo$,
  variation-values: ((at: -1, label: $2$), (at: 1, label: $-2$)),
  second-factors: (
    (expr: $6x$, zeros: ((value: $0$, approx: 0),), fn: x => 6 * x),
  ),
```

```
  second-summary-label: $f''(x)$,
  convexity: true,
  convexity-label: $f(x)$,
)
```

	x	$-\infty$		-1		0		1	$+\infty$
facteurs	$x+1$		$-$	0	$+$		$+$		$+$
facteurs	$x-1$		$-$		$-$		$-$	0	$+$
	$f'(x)$		$+$	0	$-$		$-$	0	$+$
	$f(x)$	$+\infty$	$\nearrow$	2	$\searrow$		$\searrow$	-2	$\nearrow$ $+\infty$
fact.	$6x$		$-$		$-$	0	$+$		$+$
	$f''(x)$		$-$		$-$	0	$+$		$+$
	$f(x)$		$\cap$		$\cap$		$\cup$		$\cup$

`signs` always takes precedence over `fn` if both are provided. Irrational zeros work correctly because test points are floats (e.g. for a zero at `approx: calc.sqrt(2)`, the test point in the right interval is `calc.sqrt(2) + 1.0`).

2.9. Simple table without factor strip

Set `show-facteurs: false` to hide the rotated "facteur(s)" label strip. Useful when there is only one factor row or when the factors are implied by context:

```
#sign-table(
  factors: (
    (zeros: (
      (value: $-2$, approx: -2),
      (value: $1$,  approx:  1),
    ), signs: ("+", "-", "+")),
  ),
  summary-label: $f'(x)$,
  variation: true,
  variation-label: $f(x)$,
  show-facteurs: false,
)
```

x	$-\infty$		-2		1		$+\infty$
		$+$	0	$-$	0	$+$	
$f'(x)$		$+$	0	$-$	0	$+$	
$f(x)$		↗		↘		↗	

2.10. Coloured background

Zero labels, bound labels, and variation values use a knockout rectangle to visually break the table grid lines. By default this rectangle is `white`. When the table sits on a non-white background — inside a coloured box or on a tinted page — set `background` to match so the knockout is invisible:

```
// Set once to apply to all tables in a document or scope:
#let sign-table = sign-table.with(background: luma(230))
```

All examples in this manual are rendered inside a light-grey `#f8f8f8` block. The manual sets `background` globally at the top so the labels always look correct.

2.11. Domain boundary (range not in domain)

Use `mark: "hd"` on a zero that lies at a domain boundary (not an asymptote): the hatch continues **through** the zero point, and the variation arrow starts at that edge. The zero at the boundary already appears as a column header — do **not** also set the left bound to the same value (that would create a duplicate label). Use `bounds: (left: none, ...)` instead:

```
// f(x) = sqrt(x), defined on [0, +∞). f'(x) = 1/(2√x), undefined at 0.
#sign-table(
  factors: (
    (
      expr: $frac(1, 2 sqrt(x))$,
      zeros: ((value: $0$, approx: 0, mark: "hd"),),
      fn: x => if x <= 0 { none } else { 1 / (2 * calc.sqrt(x)) },
    ),
  ),
  summary-label: $f'(x)$,
  variation: true,
  variation-label: $f(x)$,
  bounds: (left: none, right: $+oo$),
  start-value: $0$,
  start-pos: "bottom",
)
```

x		0		$+\infty$
fact. $\frac{1}{2\sqrt{x}}$			$+$	
$f'(x)$			$+$	
$f(x)$		0	↗	

A function undefined on an entire interval (e.g. $f(x) = \ln(x^2 - 4)$, undefined for $x \in]-2, 2[$) requires a domain check. Use `fn` for each factor and return `none` outside the domain. Zeros that fall inside the excluded zone (here x=0 for 2x) should not be listed in `zeros` since they do not lie in the domain of f:

```
// f(x) = ln(x²-4), domain: |x| > 2. f'(x) = 2x/(x²-4).
// x=0 is NOT listed as a zero: it is outside the domain of f.
#sign-table(
  factors: (
    (
      expr: $2x$,
      zeros: (),
      fn: x => if x * x <= 4 { none } else { 2 * x },
    ),
    (
      expr: $x^2 - 4$,
      zeros: (
        (value: $-2$, approx: -2, pole: true),
        (value: $2$,  approx:  2, pole: true),
      ),
      fn: x => if x * x <= 4 { none } else { x * x - 4 },
    ),
  ),
  summary-label: $f'(x)$,
  variation: true,
  variation-label: $f(x)$,
)
```

x	$-\infty$		-2		2		$+\infty$
facteurs $2x$		$-$				$+$	
facteurs $x^2 - 4$		$+$	0		0	$+$	
$f'(x)$		$-$	$\Vert$		$\Vert$	$+$	
$f(x)$		↘	$\Vert$		$\Vert$	↗	

3. `fun-table`

3.1. Basic usage

```
#fun-table(
  x-values: ($0$, $1$, $2$, $3$, $4$),
  rows: (
    (label: $f(x)$, values: ($1$, $3$, $5$, $7$, $9$)),
  ),
)
```

x	**0**	**1**	**2**	**3**	**4**
$f(x)$	1	3	5	7	9

3.2. Parameters

Parameter	Type	Default	Description
`x-values`	`array`	`()`	x column headers. Numbers: displayed in math mode and forwarded to `fn`. Dictionaries `(display, value)`: custom display with numeric value. Content: display-only.
`x-label`	`content`	`$x$`	Label for the x row.
`rows`	`array`	`()`	Function rows (see row keys below).
`col-width`	`length`	`1.5cm`	Column width.
`row-height`	`length`	`0.9cm`	Row height.
`label-width`	`auto`, `length`	`auto`	Label column width. `auto` sizes to fit the widest label.
`stroke`	`stroke`	`0.5pt`	Table border and separator stroke.
`decimals`	`auto`, `int`	`auto`	Decimal places for computed values. `auto` trims trailing zeros.

3.2.1. Row dictionary keys

Parameter	Type	Default	Description
`label`	`content`	—	Row label.
`values`	`array`	—	Explicit content cells. Use `none` for blank. Takes precedence over `fn` for non-numeric x.
`fn`	`function`	—	`x => number` used when `x-values` entries are numeric. Return none for a blank cell.
`format`	`function`, `none`	—	`number => content` for custom rendering. Default: smart integer/decimal.

3.3. Auto-computed values

Provide numbers in `x-values` and a `fn` closure — values are computed and formatted automatically:

```
#fun-table(
  x-values: (-3, -2, -1, 0, 1, 2, 3),
```

```
  rows: (
    (label: $f(x) = x^2 - 1$, fn: x => x * x - 1),
    (label: $g(x) = 2x + 1$,  fn: x => 2 * x + 1),
  ),
)
```

x	-3	-2	-1	0	1	2	3
$f(x) = x^2 - 1$	8	3	0	-1	0	3	8
$g(x) = 2x + 1$	-5	-3	-1	1	3	5	7

3.4. Custom display with dictionary x-values

For irrational x values or non-standard display, use `(display: ..., value: ...)` entries:

```
#fun-table(
  x-values: (
    (display: $-pi\/2$, value: -calc.pi / 2),
    (display: $0$,      value: 0.0),
    (display: $pi\/2$,  value: calc.pi / 2),
    (display: $pi$,     value: calc.pi),
  ),
  rows: (
    (label: $sin(x)$, fn: x => calc.sin(x),
     format: x => $#str(calc.round(x, digits: 4))$),
    (label: $cos(x)$, fn: x => calc.cos(x),
     format: x => $#str(calc.round(x, digits: 4))$),
  ),
)
```

x	$-\pi/2$	0	$\pi/2$	π
$\sin(x)$	-1	0	1	0
$\cos(x)$	0	1	0	-1